# RAJIV GANDHI INSTITUTE OF PETROLEUM TECHNOLOGY, JAIS, AMETHI

## Department of Computer Science and Engineering




**B.Tech. 2nd Year**

**Object Oriented Methodologies (CS224)**

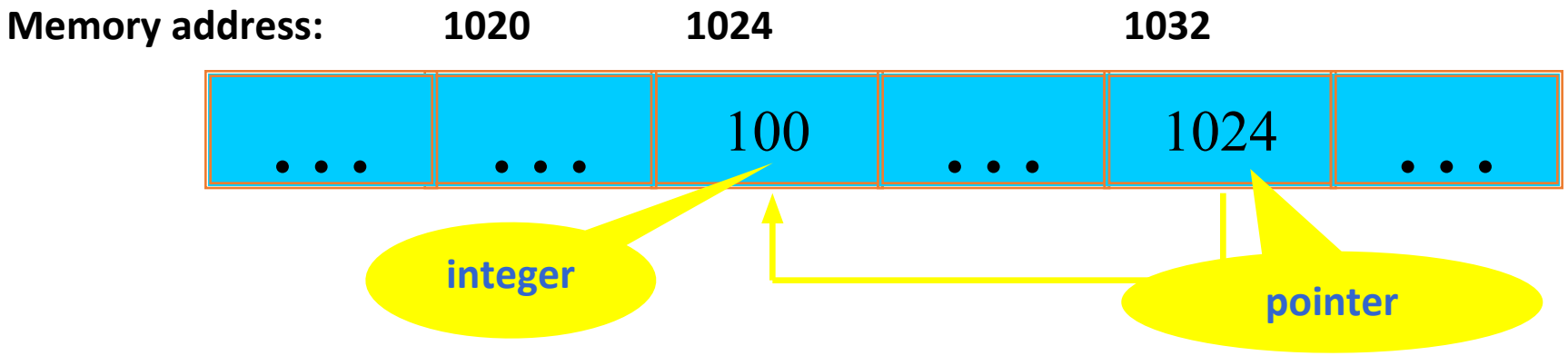
**By**

**Dr. Kalka Dubey**

# Pointers

- A pointer is a variable used to store the address of a memory cell. We can use the pointer to reference this memory cell

**datatype *var-name;**



# Operations

1. Define a pointer variables.

2. Assign the address of a variable to a pointer.

3. The value at the address available in the pointer variable and this is done by using unary operator * that returns the value of the variable located at the address specified by its operand.

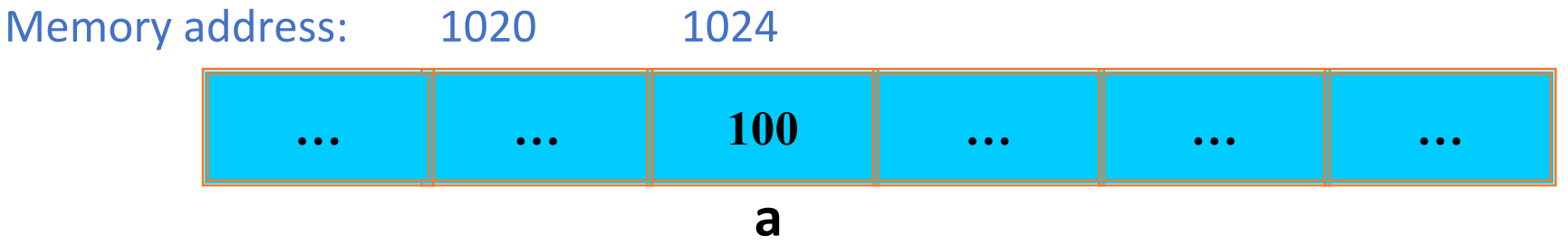
# Pointers

- Store the address of other variable

```
int x;        // int variable
int *ip;     // int pointer
ip=&x;     //address of x assigned to ip
*ip=10;   // 10 assigned to x indirectly
```

# Address operator &

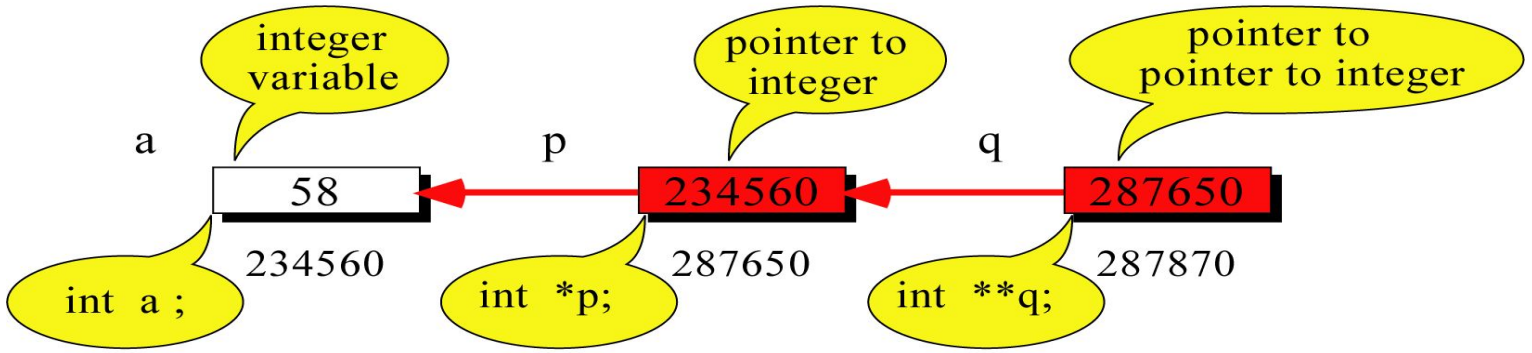
- *The "address of" operator* (&) gives the memory address of the variable




```
int a = 100;
//get the value,
cout << a;    //prints 100
//get the memory address
cout << &a;   //prints 1024
```

# Pointer to pointer

```
// Local Declarations
int         a ;
int        *p ;
int       **q ;
```

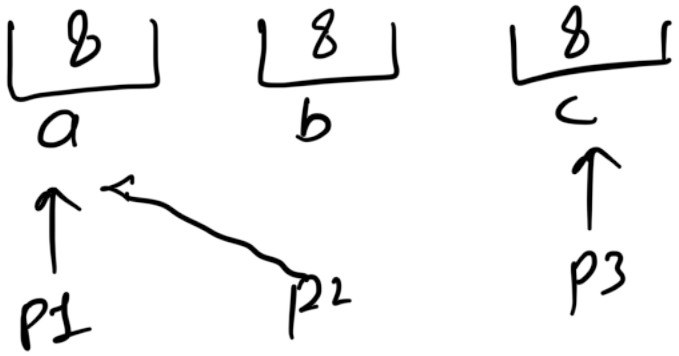


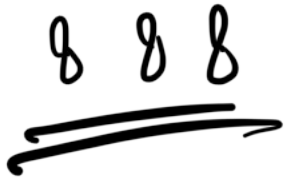

```
// Statements
a = 58 ;
p = &a ;
q = &p ;
cout <<       a << " ";
cout <<      *p << " ";
cout <<     **q << " ";
```

# Example

```
int a = 100, b = 88, c = 8;
int *p1 = &a, *p2, *p3 = &c;
p2 = &b;          // p2 points to b
p2 = p1;     // p2 points to a
b = *p3;          //assign c to b
*p2 = *p3; //assign c to a
cout << a << b << c;
```

?



# Null pointer

- A pointer that is assigned NULL is called a null pointer

```
int main ()
{
   int  *ptr = NULL;
   cout << "The value of ptr is " << ptr ;
    return 0;
}
```

To check for a null pointer you can use an if statement as follows:

- if(ptr) // succeeds if p is not null
- if(!ptr) // succeeds if p is null

# Pointer arithmetic & using arrays

- You can perform arithmetic operations on a pointer just as you can a numeric value.
- There are four arithmetic operators that can be used on pointers: ++, --, +, and –
- For example, a pointer that points to the beginning of an array can access that array by using either pointer arithmetic or array-style indexing.

# Example

```
int main ()
{
   int  var[3] = {10, 100, 200};
   int  *ptr;

   // let us have array address in pointer.
   ptr = var;
   for (int i = 0; i < 3; i++)
   {
      cout << *ptr << endl;

      // point to the next location
      ptr++;
   }
   return 0;
}
```

# C++ array of pointers

- int *ptr[MAX];

- This declares pointer as an array of MAX integer pointers.

- Thus, each element in ptr, now holds a pointer to an integer value.

# Exercise 1

```
int main ()
{
   int  var[MAX] = {10, 100, 200};
   int *ptr[MAX];

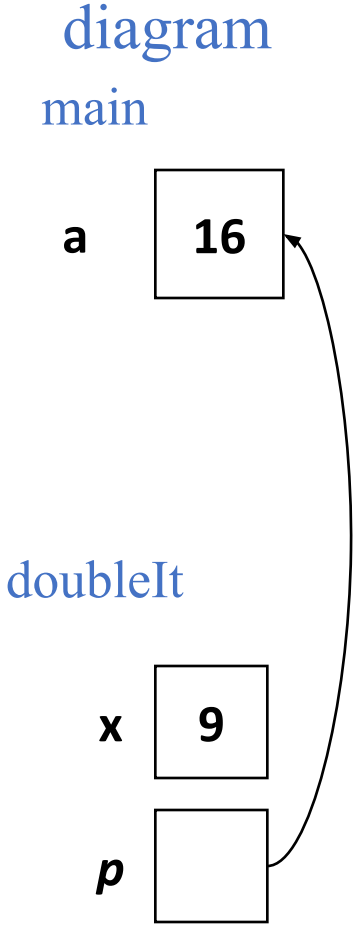
   for (int i = 0; i < MAX; i++)
   {
      ptr[i] = &var[i]; // assign the address of integer.
   }
   for (int i = 0; i < MAX; i++)
   {
      cout << "Value of var[" << i << "] = ";
      cout << *ptr[i] << endl;
   }
   return 0;
}
```

# Passing pointer to functions

The code

```
void doubleIt (int x, int * p)
{
   *p = 2 * x;
}
int main(int argc, const char * argv[])
{
   int a = 16;
   doubleIt(9, &a);
   cout<<a;
   return 0;
}
```

diagram



# Exercise 2

```
#include <iostream>
using namespace std;

int main()
{
   const int i = 20;
   const int* const ptr = &i;
   (*ptr)++;
   int j = 15;
   ptr = &j;
   cout << i;
   return 0;
}
```

Error : You cannot Modify constant values.

Modifying const pointer

# Output

Compile error

# Exercise 3

```
#include <iostream>
using namespace std;
int main()
{
   int num[5];
   int* p;
   p = num;
   *p = 10;
   p++;
   *p = 20;
   p = &num[2];
*p = 30;
   p = num + 3;
   *p = 40;
   p = num;
   *(p + 4) = 50;
   for (int i = 0; i < 5; i++)
      cout << num[i] << ", ";
   return 0;
}
```

# Output

10, 20, 30, 40, 50,

# Exercise 4

```
#include <iostream>
using namespace std;
int main()
{
   int arr[] = { 4, 5, 6, 7 };
   int* p = (arr + 1);
   cout << *arr + 10;
   return 0;
}
```

# Output

14

# Reference

- When a variable is declared as a reference, it becomes an alternative name for an existing variable.
- A variable can be declared as a reference by putting '&' in the declaration.
- If a function receives a reference to a variable, it can modify the value of the variable.
- We can use references in for each loop to modify all elements.

# Reference

```
#include <iostream>
using namespace std;

int main()
{
    int x = 10;

    // ref is a reference to x.
    int &ref = x;

    // Value of x is now changed to 20
    ref = 20;
    cout << "x = " << x << '\n';

    // Value of x is now changed to 30
```

# Output

x = 20

ref = 30

# Exercise 1

```
#include <iostream>
using namespace std;

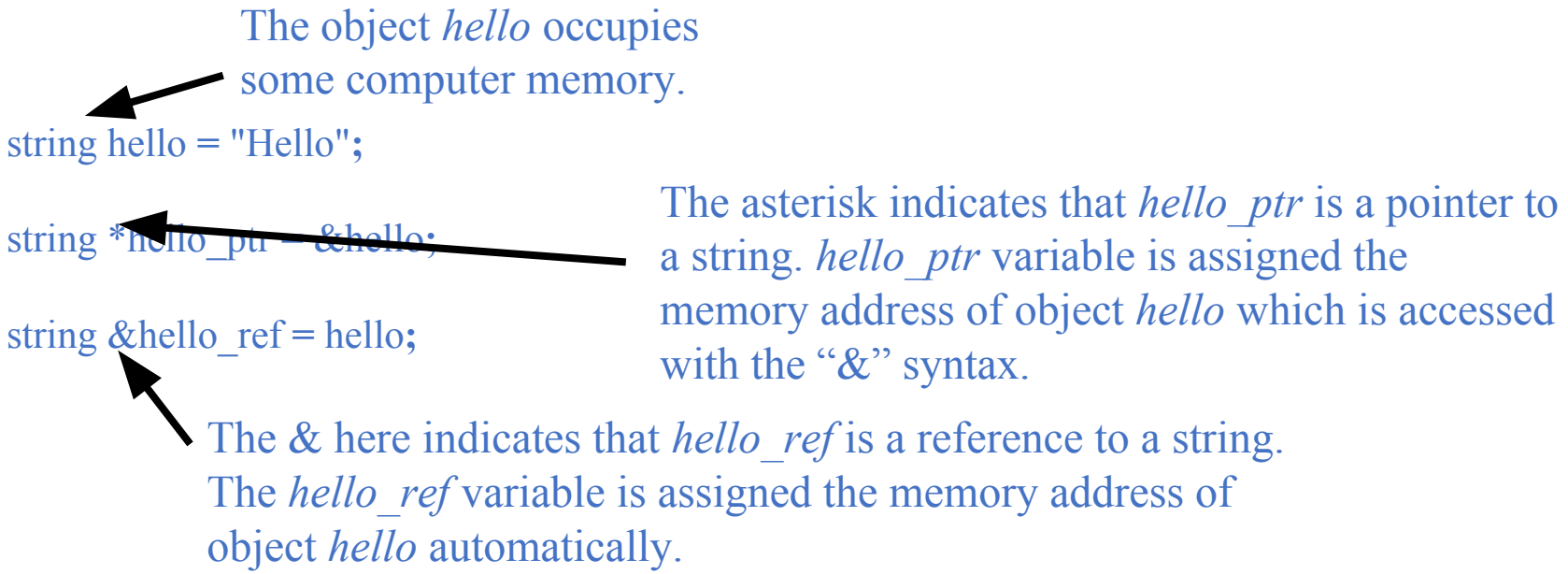
int main()
{
   int a = 32, *ptr = &a;
   char ch = 'A‘;
  char &cho = ch;

   cho += a;
   *ptr += ch;
   cout << a << ", " << ch << endl;
   return 0;
}
```

# Output

129, a

# Reference and Pointer Variables




- Variable and object values are stored in particular locations in the computer's memory.
- Reference and pointer variables **store the memory location of other variables.**
- Pointers are found in C. References are a C++ variation that makes pointers easier and safer to use.

# Difference Between Pointer and Reference

↗ A diff. variable storing address

↳ Not having any existance, just a nickname.

- A pointer can be declared as void but a reference can never be void.
- The pointer variable has n-levels/multiple levels of indirection i.e. single-pointer, double-pointer, triple-pointer. Whereas, the reference variable has only one/single level of indirection.
- Reference variable cannot be updated.
- References are less powerful than pointers.
- Declaration of a Reference variable is preceded with the '&' symbol ( but do not read it as "address of").
- References are safer and easier to use

- `p = &a;` ✓ `p = &b;` ✓ | `int &r = a;` `r = b;` ✗
- Pointers to NULL ✓ Reference to NULL ✗
- `p++, p--` ✓ `r++, r--` ✗

# Input operators

**cin >> variable-name;**

Meaning: read the value of the variable called <variable-name> from the user

Example:

```
cin >> a;
cin >> b >> c;
cin >> x;
cin >> my-character;
```

# Output operators

**cout << variable-name;**

Meaning: print the value of variable <variable-name> to the user

**cout << “any message “;**

Meaning: print the message within quotes to the user

**cout << endl;  Meaning: print a new line**

Example:

```
cout << a;
cout << b << c;
cout << “This is my character: “ << my-character << “ end ” << endl;
```

# Operators

## Arithmetic operators

| operators | met | Example |
|---|---|---|
| + | Addition | x=10,y=5<br>x+y -> 21 |
| - | Subtraction | X-y -> 5 |
| * | Multiplication | X*y -> 50 |
| / | Division | x/y -> 2 |
| % | Modulo Division | X%y -> 0 |

## Relational Operators

| Operator | Meaning | Example | Result | value |
|---|---|---|---|---|
| < | Less Than | x=10,y=5<br>X<y | False | 0 |
| > | Greater Than | X>y | True | 1 |
| <= | Less than or Equal to | X<=y | False | 0 |
| >= | Greater than or equal to | X>=y | True | 1 |
| != | Not Equal to | X!=y | False | 0 |
| == | Equal To | X==y | False | 0 |

# Operators

## Logical Operators

| operators | met | Example |
|---|---|---|
| && | Called Logical AND operator. If both the operands are non-zero, then condition becomes true. | Assume variable A holds 1 and variable B holds 0<br><br>(A && B) is false |
| \|\| | Called Logical OR Operator. If any of the two operands is non-zero, then condition becomes true. | (A \|\| B) is true |
| ! | Multiplication | !(A && B) is true |

## Assignment Operators

| Operator | Meaning | Example |
|---|---|---|
| = | Simple assignment operator | C = A + B |
| += | Add AND assignment operator | C += A |
| -= | Add AND assignment operator | C -= A |
| *= | Multiply AND assignment operator | C *= A |
| /= | Divide AND assignment operator | C /= A |
| %= | Divide AND assignment operator | C %= A |

# Others Operators

- Scope resolution operator ::
- Member dereferencing operator ->, *, ::*
- Memory management operator malloc(), sizeof(), new, delete
- Type cast operators type_name (expression)

# Scope resolution operator → used to access Global Variable.

```
#include <iostream>
using namespace std;
int num = 50;
int main ()
{
int num = 100;

cout << " The value of the local variable num: " << num;
cout << "\n The value of the global variable num: " << ::num;
return 0;
}
```

# Output

The value of the local variable num: 100

The value of the global variable num: 50

# Member dereferencing operator

- **Pointer to a member declarator ( ::* ) :** To declare a pointer to a member of a class.

- **Pointer to member operator( .* ) :** To access a member using an object name and a pointer to that member.

- **Pointer to member operator ( ->* ) :** To access a member using a pointer to the object and a pointer to that member.

# Dynamic variable using new

```
#include <iostream>
#include <memory>
using namespace std;
int main()
{
int* ptr;
ptr = new int;
*ptr = 10;
  cout << "Address: " << ptr << endl;
  cout << "Value: " << *ptr;
    return 0;
}
```

# Dynamic variable using new

```
#include <iostream>
using namespace std;
int main()
{
    int* arr = new int[5];
  cout << "Enter 5 values in the array: ";
    for(int i = 0; i < 5; i++)
  {
    cin >> arr[i];
  }
  cout << "\nArray elements: ";
  for(int i = 0; i < 5; i++)
  {
    cout << arr[i] << " ";
  }
  delete [] arr;
    for(int i = 0; i < 5; i++)
    {
      cout << arr[i] << " ";
    }

    return 0;
}
```

# Type Casting

Used to convert one datatype into another data type.

```
#include<iostream.h>
#include<conio.h>
void main()
{
int a=10, b;
float c=2.3;
clrscr();
b=int (a+c);
Cout<<"The value of b is <<b;
getch();
}
```

# Control structures

- if
  - if else
  - Nested if
- Switch
- do while
- While
- for

Working of if..else statement when test condition is true

```
int test = 5;

if ( test < 10 ) {

    // some code
    // some code
}

else {

    // some code
    // some code
}

// code just below if..else
```

Working of if..else statement when test condition is false

```
int test = 5;

if ( test > 10 ) {

    // some code
    // some code
}

else {

    // some code
    // some code
}

// code just below if..else
```

# Loops

- For loop

```
for(initialization statement; test expression; update statement)
{
codes to be executed;
 }
```

- While loop

```
while (test expression)
{
statement/s to be executed.
 }
```

# Loops

- Do while

```
do
 {
statement/s;
 } while (test expression);
```

# Loops Example

- Write a C++ Program for Fibonacci Series.

# Loops Example

```
#include<iostream>
using namespace std;
int main()
{
    int a=0,b=1,t,j=1,num;
    cout<<"enter the limit ";
    cin>>num;
    cout<<"The Fb Series is "<<a<<b;
    do
    {
        t=a+b;
        cout<<t;
        a=b;
        b=t;
```

# Loops Example

Output

enter the limit 3

The Fb Series is 01123.

# Loops Example

- Write a program to print all Armstrong numbers between 1 and 500.

# Loops Example

```
// If sum of cubes of each digit of the number is equal to the number itself, then the number is called an Armstrong number.
#include<iostream>
using namespace std;

int main()
{
	int n,digit1,digit2,digit3;

	for(int i=1;i<=500;i++)
	{
		digit1=i/100;
		digit2=i/10 - digit1*10;
		digit3=i%10;

		if (digit1*digit1*digit1 + digit2*digit2*digit2 + digit3*digit3*digit3 == i)
			cout<<i<<endl;
	}


	return 0;
}
```

# Loops Example

Output

1
153
370
371
407.

# Loops Example

- Write a program to calculate HCF of Two given number.

# Loops Example

```
#include<iostream>
using namespace std;
int main()
{
	int dividend, divisor, rem=1, hcf;
	cout<<"Enter two numbers : ";
	cin>>dividend>>divisor;
	while(rem!=0)
	{
		rem=dividend % divisor;
		if(rem==0)
			hcf=divisor;
		else
		{
			dividend=divisor;
			divisor=rem;
		}
	}
	cout<<"HCF is : "<<hcf;
	return 0;
}
```

# Loops Example

Output

Enter two numbers : 30 105

HCF is : 15

# Switch

```
switch (n) {
case constant1:
code/s to be executed if n equals to constant1;
break;
case constant2:
code/s to be executed if n equals to constant2;
break; . .
.
default:
code/s to be executed if n doesn't match to any cases;
 }
```

# Functions

The return type is *float*.

The function arguments L and W are sent as type *float*.

```
float RectangleArea1(float L, float W) {
   return L*W ;
}


float RectangleArea2(const float L, const float W) {
   // L=2.0 ;
   return L*W ;
}

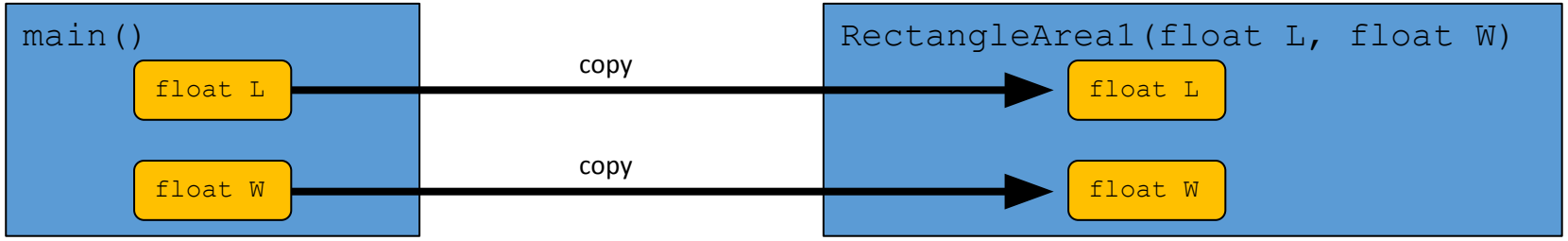

float RectangleArea3(const float& L, const float& W) {
   return L*W ;
}


void RectangleArea4(const float& L, const float& W, float& area) {
   area= L*W ;
}
```

Product is computed

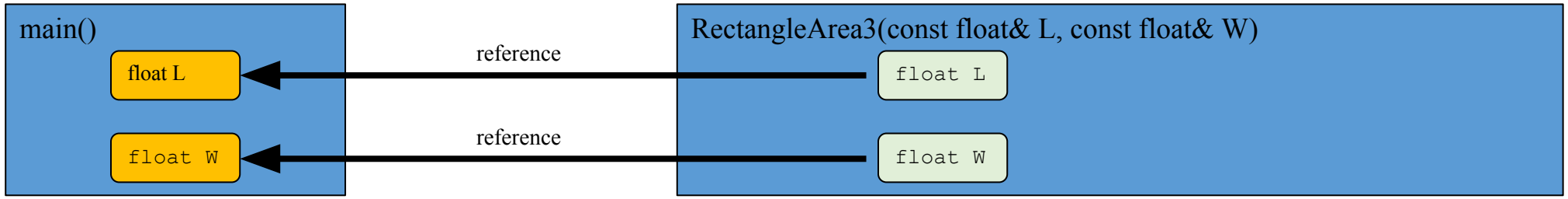


# Pass by Value




- C++ defaults to *pass by value* behavior when calling a function.
- The function arguments are **copied** when used in the function.
- Changing the value of L or W in the RectangleArea1 function does **not** effect their original values in the main() function
- When passing objects as function arguments it is important to be aware that potentially large data structures are automatically copied!

# Pass by Reference




- *Pass by reference* behavior is triggered when the & character is used to modify the type of the argument.
- This is the type of behavior you see in Fortran, Matlab, Python, and others.
- Pass by reference function arguments are **NOT** copied. Instead the compiler sends a *pointer* to the function that references the memory location of the original variable. The syntax of using the argument in the function does not change.
- Pass by reference arguments almost always act just like a pass by value argument when writing code except that changing their value changes the value of the original variable!!
- The *const* modifier can be used to prevent changes to the original variable in main().

# Function Example

- Write a function that receives two numbers as an argument and display all prime numbers between these two numbers. Call this function from main( ).

# Function Example

```
#include<iostream>
using namespace std;

void showprime(int, int);

int main()
{
	int x,y;
	cout<<"Enter first  number : ";
	cin>>x;
	cout<<"Enter second  number : ";
	cin>>y;
	showprime(x,y);
	return 0;
}
void showprime(int a, int b)
{
	bool flag;
	for(int i=a+1;i<=b;i++)
	{
```

# Function Example

**Output**

```
Enter first number : 15
Enter second number : 70
17
19
23
29
31
37
41
43
47
53
59
61
67
```

# Function Example

- Write a function called zero_small() that has two integer arguments being passed by reference and sets the smaller of the two numbers to 0. Write the main program to access the function.

# Function Example

```
#include<iostream>
using namespace std;
void zero_small(int &,int &);
int main()
{
    int x,y;
    cout<<"Enter first number : ";
    cin>>x;
    cout<<"Enter second number : ";
    cin>>y;
    zero_small(x,y);
    cout<<"First number is : "<<x;
cout<<"\nSecond number is : "<<y;
```

# Function Example

## Output

Enter first number : 23

Enter second number : 6

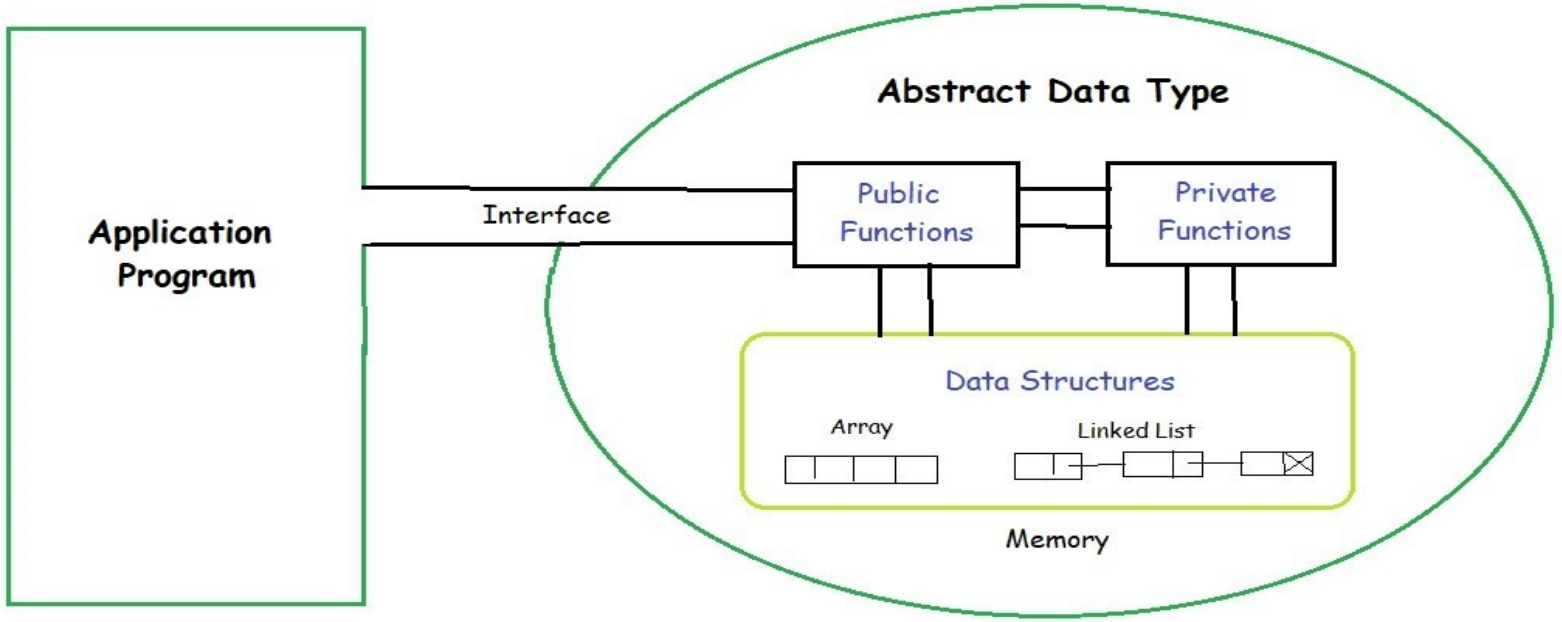
First number is : 23

Second number is : 0

# Abstract Data Types

- Abstract Data type (ADT) is a type (or class) for objects whose behavior is defined by a set of values and a set of operations.
- The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented.
- It is called “abstract” because it gives an implementation-independent view.
- The process of providing only the essentials and hiding the details is known as abstraction.

# Abstract Data Types



# Abstract Data Types in C++

- Data abstraction allows a program to ignore the details of how a data type is represented.
- Abstraction (derived from a Latin word abs, meaning away from and traverse, meaning to draw) refers to the act of representing essential features without including the background details or explanations.
- C++ classes use the technique of abstraction and are defined as a list of abstract attributes such as width, cost, size etc and functions to operate on these attributes.
- They put in a nutshell all the essential properties of an object that are required to be created.
- The attributes are therefore called data members as they hold information.
- The functions that operate on those data members are termed member functions.
- Since C++ classes use the data abstraction concept, they are also termed abstract data types.

# THANKS…..